

DEPLOY_CLOUD_LINUX_ONLY

NetworkPeer — Demo From the Linux Laptop ONLY (No Mac in the Office)

THIS IS THE REQUIRED PATH (updated Aug 11): the Mac will NOT be in the office, so the tunnel approach is dead. Everything must run in the cloud; the office Linux laptop only opens URLs in a browser. This guide uses **demo mode** (see the warning below) so the whole flow — sign-up, funding, evidence, approval — works tonight without Twilio or Stripe accounts (OTP is shown on the verify screen; payments are simulated).

The important trade-off (read this first)

A production-mode deploy rejects `OTP_ECHO_IN_RESPONSE=true`, `SMS_PROVIDER=console` and `PAYMENT_GATEWAY=stub` — which means it needs a **Twilio** account for login SMS and Stripe keys for payments. We don't have those, so this guide runs the API with `NODE_ENV=development` on the cloud host. Consequences, all acceptable for a one-hour demo:

- OTP codes are shown on the login screen (any phone number works).
- Payments are simulated via the stub gateway + a browser webhook-settler page.
- **Do not** put real customer data on this deployment, and revert to production config (`EXECUTIVE_PRESENTATION_AND_DEPLOYMENT_GUIDE.md` §2.5) before any real launch.

Architecture

Linux laptop (browser only)
| `https://<vercel-origin>`

Vercel (frontend, built

```

from GitHub)
  | https://<api>.up.railway.app      Railway (API service,
NODE_ENV=development)              Railway (PostgreSQL +
  |                                Railway (Redis)
PostGIS)                           AWS S3 eu-north-1
  |                                https://<vercel-
  |                                origin>/dev/settle-funding
(evidence)
  |
origin>/dev/settle-funding

```

Step 0 — Prereqs (10 min)

1. GitHub repo already has everything (`addy9087/Networkpeer` , pushed).
2. AWS: finish creating the S3 bucket (see the AWS notes at the bottom of this doc), create an IAM user with keys, add them to the Railway API env later.
3. Sign up for **Railway** (railway.app) and **Vercel** (vercel.com) with GitHub login. No card needed for Railway's trial credits / Vercel hobby.

Step 1 — Database on Railway (5 min)

1. Railway → New Project → "Provision PostgreSQL" → name it `networkpeer-db` . Open it → **Variables** tab → copy the `DATABASE_URL` (the public proxy URL).
2. Add a second plugin: "Provision Redis" → copy its `REDIS_URL` (public).
3. The public Postgres URL works with TLS via `sslmode=require` — this avoids Railway's private-hostname restriction entirely.
4. Migrate + create the app role (run once from the Mac, using the Postgres URL):

```

cd /Users/adityasharma/Desktop/NETWORKPEER/NetworkPeer-main
export DATABASE_URL="postgresql://postgres:PASSWORD@host.up.railway.app:PORT/railway?
    sslmode=require"
npm run migrate
psql "$DATABASE_URL" -v ON_ERROR_STOP=1 -f scripts/provision-
    app-role.sql

```

(Run the provisioning with the four `NETWORKPEER_*_DB_PASSWORD` env vars exported; the script reads them. In demo mode you may instead use the owner URL directly as `DATABASE_URL` and leave the other three empty — simplest for the demo.) > Free alternative: **Neon** (console.neon.tech) — same migrate/provision commands against > its `?sslmode=require` URL. PostGIS is supported.

Step 1.5 — How the database is visible (how to see your data)

There are two sides to “seeing” the DB:

How the API connects (invisible to you, but worth knowing): the API only knows the DB through the `DATABASE_URL` env var (Step 1). The migrations + role provisioning run against that same URL. No other config touches the DB.

How YOU view the data — three ways:

1. **Railway dashboard (easiest, no tools):** open the `networkpeer-db` Postgres service → **Data** tab → pick a table (e.g. `users` , `jobs`) from the left list to browse rows, or click **Query** for a SQL editor (run `SELECT * FROM jobs;`). Table + column names come from the migrations (`NetworkPeer-main/migrations/*.sql`).
2. **psql from the Mac** (or any machine with `psql` installed):

```
psql "postgresql://postgres:PASSWORD@host.up.railway.app:PORT/railway?
sslmode=require"
# then: \dt          (list tables)
#       \d jobs      (describe a table)
#       SELECT * FROM schema_migrations ORDER BY filename DESC
#               LIMIT 3;
```

3. **Neon console** (if you used Neon instead): `console.neon.tech` → project → **SQL Editor** — same queries, no local install.

Local Mac tonight (while testing): the Docker PostGIS is at `localhost:5433` — see `TEST_IT_NOW_AND_TOMORROW.md` §5 (`docker exec` , `npm run db:shell` , or any Postgres GUI pointed at `localhost:5433` , user `postgres`). The cloud DB is the same schema, just remote.

Step 2 — API on Railway (15 min)

1. Railway → New Project → **Deploy from GitHub** → select `addy9087/Networkpeer` .
2. Create a service from the repo with **Root directory** = `NetworkPeer-main` .
3. **Variables** (set all of these; generate secrets with `openssl rand -hex 48`):

```

NODE_ENV=development
API_PREFIX=/api/v1
DATABASE_URL=postgresql://...?sslmode=require      (from Step 1)
DATABASE_ADMIN_URL=                                (empty =
demo mode uses DATABASE_URL)
DATABASE_MEDIA_VERIFIER_URL=
DATABASE_FINANCIAL_URL=
REDIS_URL=redis://... or rediss://...              (from Step
1)
JWT_SECRET=<openssl rand -hex 48>
JWT_REFRESH_SECRET=<openssl rand -hex 48>
JWT_ACCESS_TTL=15m
JWT_REFRESH_TTL=7d
JWT_ISSUER=networkpeer-api
JWT_AUDIENCE=networkpeer-mobile
OTP_ECHO_IN_RESPONSE=true
SMS_PROVIDER=console
PAYMENT_GATEWAY=stub
PAYMENT_WEBHOOK_SECRET=<openssl rand -hex 48>      ← MUST match
the frontend build var below
PLATFORM_FEE_BPS=0
CORS_ORIGINS=https://<vercel-origin>              (add after
Step 3, then redeploy)
BACKGROUND_QUEUES_ENABLED=true
REALTIME_ENABLED=true
AWS_REGION=eu-north-1
AWS_S3_BUCKET=<your-bucket>
AWS_ACCESS_KEY_ID=<iam-key>
AWS_SECRET_ACCESS_KEY=<iam-secret>
AWS_SESSION_TOKEN=                                (empty)
LOG_LEVEL=info
LOG_PRETTY=false
RATE_LIMIT_WINDOW_MS=60000
RATE_LIMIT_MAX_REQUESTS=100

```

(Railway sets `PORT` itself.)

4. **Settings → Deploy** → Build command: `npm ci && npm run build` →
Start command: `node dist/index.js` .
5. Verify: open `https://<your-service>.up.railway.app/api/v1/live` →
`{"success":true,...}` and `/api/v1/health` → `database: true` .
6. Worker service (optional for the demo): a second service, same env, start
command `node dist/background-worker.js` ,
`BACKGROUND_QUEUES_ENABLED=true` . The demo works with the API-only
default (queues run inside the API process in demo mode).

Step 3 — Frontend on Vercel (10 min)

1. vercel.com → New Project → import `addy9087/Networkpeer` → **Root directory** = `NetworkPeer-platform-main` .
2. Framework preset: other (Nitro). Settings:

- **Node.js 24**
- Build command: `npm run build`
- Environment variables:

```
NITRO_PRESET=vercel
VITE_API_BASE_URL=https://<your-api>.up.railway.app/api/v1
VITE_API_PREFIX=/api/v1
VITE_DEMO_WEBHOOK_SECRET=<same PAYMENT_WEBHOOK_SECRET as Step 2>
```

3. Deploy → copy the exact `https://<vercel-origin>` URL → set `CORS_ORIGINS` on the Railway API to that origin and **redeploy the API service**.
4. Verify: open the Vercel URL → landing page loads → `https://<vercel-origin>/dev/settle-funding` shows the funding settler.

Realtime note: Vercel serverless doesn't support long-lived Socket.IO connections, so live toasts may not fire — job statuses still update correctly on refresh (durable REST sync). If realtime is essential, host the frontend on Railway too (start: `node .output/server/index.mjs` , with `NITRO_PRESET=node-server` at build) instead of Vercel.

Step 4 — S3 (your account, already live)

- Bucket settings (you're on the Create screen now): **Region eu-north-1 (Stockholm)**, General purpose, **Block all public access = ON**, **Versioning = Enable** (currently shows Disable — switch it), **Default encryption = SSE-S3**, no tags → **Create bucket**.
- IAM: create a user (Access key type: Programmatic) and attach an inline policy. The policy **name is arbitrary** (nothing reads it; the current one was saved as `new`):

```
{
  "Version": "2012-10-17",
  "Statement": [
```

```
{ "Effect": "Allow", "Action": ["s3:PutObject",
    "s3:GetObject", "s3:PutObjectTagging"],
  "Resource": "arn:aws:s3:::<bucket>/*" },
{ "Effect": "Allow", "Action":
  ["s3:GetBucketPublicAccessBlock",
    "s3:GetEncryptionConfiguration",
    "s3:GetBucketVersioning", "s3:PutBucketCORS",
    "s3:ListBucket"],
  "Resource": "arn:aws:s3:::<bucket>" }
]
```

- Copy the keys into the Railway API env (Step 2).
- Bucket CORS — allowed origin must be the **Vercel origin** (and localhost for Mac testing):

```
cat > /tmp/networkpeer-s3-cors.json <<EOF
{ "CORSRules": [ { "AllowedHeaders": ["*"], "AllowedMethods":
  ["POST"],
  "AllowedOrigins": ["http://localhost:8080",
    "https://<vercel-origin>"],
  "ExposeHeaders": ["ETag"], "MaxAgeSeconds": 300 } ] }
EOF
aws s3api put-bucket-cors --bucket "$AWS_S3_BUCKET" --cors-
configuration file:///tmp/networkpeer-s3-cors.json
```

No aws CLI? Use the repo verifier instead — it sets CORS to
localhost:8080 plus any extra origins you pass:

```
cd NetworkPeer-main && node scripts/verify-s3-setup.mjs
https://<vercel-origin>
```

Step 5 — Tomorrow on the Linux laptop (browser only)

1. Normal window → Vercel URL (client). Incognito window → same URL (worker).
2. Sign up both roles — the OTP appears on the verify screen (demo mode).
3. Provision the admin + verify the worker via SQL against the cloud DB (from the Mac, or any machine with `psql`):

```
psql "postgresql://postgres:PASSWORD@host.up.railway.app:PORT/railway?
sslmode=require" \
-f scripts/provision-admin.sql
psql "postgresql://postgres:PASSWORD@host.up.railway.app:PORT/railway?
sslmode=require" \
```

```

--set=admin_phone="+1XXXXXXXXXX" --
set=worker_phone="+1XXXXXXXXXX" <<'SQL '
SELECT public.admin_set_worker_verification(
  (SELECT id FROM public.users WHERE phone_number =
    :admin_phone'),
  (SELECT id FROM public.users WHERE phone_number =
    :worker_phone'),
  'VERIFIED', TRUE, 'Verified for live demonstration');
SQL

```

4. Client creates the job (checklist count + map picker), clicks **Fund escrow**, then opens `https://<vercel-origin>/dev/settle-funding`, pastes operation ID + provider reference, clicks **Settle funding** → job becomes **POSTED**.
5. Worker finds the job nearby, accepts, runs the task statuses, uploads evidence (real S3), submits. Client approves → wallet shows the full payout (fee 0).
6. Keep the demo offline-safe: if the corporate network blocks outbound HTTPS to Cloudflare/ Railway/Vercel, use a phone hotspot instead.

After the demo (cleanup / production-ify)

- Delete or lock the Railway/Vercel demo deployments, or keep them behind proper secrets.
- To go real: set `NODE_ENV=production`, add Stripe keys, disable OTP echo, set real 4-role database URLs, and remove `VITE_DEMO_WEBHOOK_SECRET` from the frontend build. (Twilio was intentionally skipped for this demo — real SMS requires a paid Twilio plan; console-mode OTP echo is what makes the demo work for free.)